# AI DOC

## Topics:

- Learning the behind the scenes of the command line
- Game design principles

# The Command Line

# Introduction

3 pieces to a terminal

- terminal emulator
  - only handles text, input, colors, and scrolling
  - does not know what the commands mean
- Shell
  - reads and interprets the command given
- Operating System (Kernal)
  - The shell asks the OS to:
    - create processes, allocate memory, access files, and communicate with hardware

# Introduction (cont.)

Main operation loop: Input -> Interpret -> Execute -> Output

The terminal is very powerful because:

- All the commands give you precise control over the computer
- You can create scripts to automate processes
- Having no UI increases the speed
- Have access to SSH
  This is where the terminal excells over gui's

## How Shells Parse Commands

The shell does not just split the command by spaces, it goes through multiple more phases:

1. Lexing or Tokenization
    i. tokens are separated by spaces, recognizes special characters, recognizes quotes

# How Shells Parse Commands (cont.)

2. Parsing
    i. In this step a syntax tree is made
    ii. The grammer is seen as:
        a. command, pipeline, redirect, conditional execution, subshells
        b. ex:

```
redirect(    pipeline(        command(echo, $HOME),
command(grep, user)    ),    out.txt )
```

        c. This Structure is referred to an Abstract Syntax Tree

# How Shells Parse Commands (cont.)

3. Expansions
    i. The shell performs expansions before it executes in this order:
        a. brace expansion: `echo file{1..3}` -> echos files 1 - 3
        b. Tilde expansion: `echo ~` -> echo /home/you
        c. parameter expansion: `echo $HOME` -> echos the HOME variable
        d. command substitution: `echo $(date)` -> echos the result of the command date
        e. arithmetic expansion: `echo $((2 + 3))` -> echos 5
        f. Word Splitting: Splits words by spaces, unless quoted
        g. globbing, or filename expansion: `echo *.txt` echos all files that end in .txt

## How Shells Parse Commands (cont.)

4. Redirections
    i. Before execution, the shell sets up file descriptors
5. Execution
    i. decides whether the command is a function, built-in, or external
    ii. runs

# Game Design

Game design compiles psychology, storytelling, art, and systems thinking to engage players. Create a meaningful experience for the player.

# Principles

1. Player-Centered Design
    i. How should the player feel
    ii. Is the game intuitive/confusing
    iii. Is it fun/frustrating
    iv. Playtesting is important
2. Create a Core Gameplay Loop
    i. Example: Action → Reward → Upgrade → Repeat
    ii. Good loops keep players engaged
        a. simple, but fun to repeat

3. Balance
    i. An unfair game does not feel fun
    ii. There is a difference between difficulty and unfairness
    iii. Too easy created boredome
    iv. Flow is a psychology concept about keeping players in between boredom frustration
4. Clear goals and Feedback
    i. It is important for a player to know what they can and should be doing
        a. How well they are doing
    ii. Some Feedback examples are
        a. sound effects
        b. visual cues
        c. progress indicators

5. Meaningful choices
    i. trade-offs - gain one thing, lose another
    ii. multiple strategies
    iii. choices that affect the outcome

6. Progression and rewards
    i. Examples: Levels, skills, upgrades, better gear, narrative progression
7. simplicity
    i. easy to learn, but still has depth
8. Immersive
    i. cohesive theming

## Game Design tips I received:

1. Be wary of the user having to repeat commands
    i. This can become tedious to have to retype, especially if the command is fairly long
    ii. Some solutions:
        a. Scripts/Macros/Aliases to shorten user input. Also command history
        b. Make commands more expressive so that players have to think about them more
        c. Pressure: Enemies continue to move and mistyping a command can have consequences

2. Make the learning of the language feel like mastery of it
3. Reward for creative use of commands
4. Good Feedback
    i. Make sure to have visual and/or audible feeback
5. Add Variant loops
    i. Combat that focuses on fast decisions
    ii. Exploration
    iii. Puzzles
    iv. Have reasource management

6. Make sure the story/narrative has reason for the command line
7. Decide whether the game should be fast or slow paced
8. Suggestion given: command chaining. Not sure if applicaple now, but may be interesting in the future.

## I asked about a way to make the combat to be more interesting:

1. change commands from move foreword or attack to dash foreword, lunge goblin, circle enemy, retreat west, guard
    i. decrease typing and increase meaning
2. Intent based combat
    i. decision making instead of just typing fast

3. commands should persist, not be one off
    i. define behavior
    ii. have commands last for a duration
    iii. otherwise players will spam
4. Cooldown/commitment
5. Enemies should demand different commands to defeat